Hackathon

Panoramica, Tecnologie e patterns.

Il progetto, sviluppato in vista dell’esame di Applicazioni Web e Mobile, è ispirato alla consegna d’esame assegnata per il progetto di Ingegneria del Software 2025-2026, che prevedeva la realizzazione di un software per la gestione di un hackathon.

Il progetto è sviluppato individualmente da Aziz Gazzella, matricola 118533, responsabile della progettazione e dello sviluppo sia del frontend sia del backend. Nonostante sia stata presa come base la consegna di Ingegneria del Software, il progetto, backend incluso, è stato risviluppato per le seguenti motivazioni: inizialmente, la versione in Spring Boot non era ottimizzata e, purtroppo, non era all’altezza dell’ecosistema in cui doveva essere integrata. In secondo luogo, per seguire le lezioni, ho optato per utilizzare le principali tecnologie mostrate, tra le quali Express.js e Node.js, adattando i pattern ai requisiti specifici del progetto di Applicazioni Web e Mobile.

Backend

Il backend del progetto ha il compito di essere indipendente dal frontend, permettendone, se necessario, l'implementazione libera dai vincoli del frontend in una qualunque altra versione futura dell'applicativo, come un'app mobile, garantendo il riutilizzo delle funzionalità sviluppate.

Il backend è sviluppato in JavaScript, reso eseguibile al di fuori del browser tramite Node.js, e utilizza il framework Express per gestire le chiamate HTTP e le API REST, sulle quali ci soffermeremo più avanti. SQLite funge da database relazionale, interrogato tramite SQL. L'architettura separa Controller, Service e Repository e include middleware per l'autenticazione, l'autorizzazione e la gestione degli errori.

Design Patterns & Responsabilità

I design pattern presenti nel progetto di Ingegneria del Software sono stati mantenuti e riorganizzati, distribuendo le responsabilità tra Adapter, Controller, Service, State, Repository e Strategy. Sono stati inoltre aggiunti i middleware, un meccanismo di Express che permette di effettuare controlli più approfonditi. La vista, ovviamente, è gestita separatamente dal backend, all'interno del frontend.

Controller

Si occupa di ricevere le chiamate HTTP, recuperare i dati ricevuti dal frontend, richiamare il Service appropriato e restituire la risposta, ponendosi tra le API e il livello applicativo.

Service

Contiene la logica di business dei singoli componenti, suddivisa per responsabilità, coordinando le operazioni che collegano diverse Repository e Adapter tra loro.

Repository

Gestisce l'accesso al database (SQLite) tramite query, fornendo le operazioni necessarie, sia di interrogazione sia di popolamento o eliminazione dei dati.

Integrità e sicurezza

Il backend è sviluppato in modo da rispettare criteri di integrità e sicurezza dei dati, inclusa la gestione delle password.

Per quanto riguarda la sicurezza, viene utilizzato il modulo crypto di Node.js, che mette a disposizione funzioni crittografiche per proteggere le credenziali degli utenti. In particolare, le password vengono elaborate tramite scrypt, generando un hash associato a un salt casuale, evitando così di salvarle in chiaro nel database. Durante il login, la password inserita viene verificata confrontando il risultato dell'elaborazione con quello memorizzato.

L'integrità dei dati viene invece garantita attraverso diversi meccanismi. Le query SQL utilizzano parametri separati dai comandi, riducendo il rischio di SQL injection. Il database SQLite utilizza inoltre chiavi esterne e vincoli di integrità per mantenere coerenti le relazioni tra le entità. Le operazioni che coinvolgono più modifiche ai dati possono essere gestite tramite transazioni, evitando che un'operazione incompleta lasci il database in uno stato incoerente.

Infine, il backend verifica i permessi dell'utente e la proprietà delle risorse prima di consentire determinate operazioni, impedendo, ad esempio, a un organizzatore di modificare hackathon appartenenti ad altri utenti.

Middleware

Vorrei aggiungere una specificazione sui middleware precedentemente menzionati: questi consentono di eseguire controlli durante la gestione delle richieste HTTP, prima che raggiungano il controller. Nel progetto vengono utilizzati per verificare la validità della sessione tramite Bearer Token, controllare i ruoli dell'utente e gestire gli errori, restituendo risposte coerenti con il tipo di problema riscontrato.

Ulteriori dettagli del backend

Il backend prevede una separazione tra la connessione al database, la gestione dello schema e il popolamento dei dati dimostrativi. Il file connection.js crea la connessione a SQLite, mentre schema.js si occupa della creazione delle tabelle e delle migrazioni. Il file seed.js gestisce invece i dati dimostrativi, consentendone il popolamento esclusivamente negli ambienti di sviluppo e test.

Per la gestione degli errori HTTP è stato realizzato un modulo dedicato, che definisce i codici di stato ammessi e permette di associare un codice specifico a ciascun errore. Infine, sono presenti implementazioni simulate degli adapter, come FakeCalendarAdapter e FakePaymentAdapter, che consentono di testare alcune funzionalità senza ricorrere a servizi esterni reali.

Frontend

Il frontend del progetto è sviluppato utilizzando Angular e Bootstrap. Permettono rispettivamente, il primo, di organizzare l'interfaccia in componenti e gestire la navigazione tra le diverse sezioni dell'applicazione, mentre Bootstrap viene utilizzato per la parte grafica e per adattare il layout alle diverse dimensioni dello schermo.

Il codice dell'applicazione è scritto in TypeScript, mentre HTML e CSS vengono utilizzati rispettivamente per definire la struttura e personalizzare l'aspetto delle pagine.

Per comunicare con il backend viene utilizzato HttpClient di Angular, attraverso il quale il frontend invia richieste HTTP alle API e riceve i dati necessari per mostrare le informazioni all'utente. La gestione delle risposte asincrone si basa anche su RxJS.

Il frontend è separato dal backend e non accede direttamente al database: le operazioni sui dati vengono richieste al server, che si occupa della logica applicativa e della persistenza tramite SQLite.

Comunicazione con il backend

Il frontend comunica con il backend attraverso richieste HTTP alle API REST, utilizzando HttpClient di Angular. L'indirizzo delle API viene definito tramite la proprietà apiUrl nei file di configurazione dell'ambiente, impostata su /api, così da mantenere separate le chiamate HTTP dalla destinazione effettiva del server.

Proxy e CORS

Durante lo sviluppo viene utilizzato un proxy API, configurato per inoltrare le richieste dirette al percorso /api verso il backend Express, in esecuzione sulla porta 3000. Questa soluzione permette di gestire la comunicazione tra frontend e backend senza dover abilitare il CORS per le richieste locali effettuate attraverso il proxy.

Organizzazione dei componenti e navigazione

Il frontend di HackHub adotta un'organizzazione basata su componenti e routing, che consente di suddividere l'interfaccia in elementi distinti e di gestire la navigazione tra le diverse pagine dell'applicazione.

Il template principale, definito in app.html, costituisce la struttura condivisa dell'interfaccia. Al suo interno sono presenti la barra di navigazione, il logo della piattaforma, i collegamenti alle diverse sezioni e gli elementi relativi all'utente autenticato, come il ruolo, l'indirizzo email e il pulsante di logout. La visualizzazione di alcuni elementi dipende dallo stato di autenticazione e dal ruolo dell'utente, permettendo di adattare il menu alle funzionalità disponibili.

Il contenuto specifico delle pagine viene visualizzato attraverso il componente router-outlet, che rappresenta il punto di inserimento del componente associato alla rotta attiva. In questo modo, la struttura comune dell'applicazione rimane separata dal contenuto delle singole sezioni.

La configurazione delle rotte è definita in app.routes.ts, dove ogni percorso URL viene associato al relativo componente. Tra le principali sezioni disponibili figurano l'elenco degli hackathon, il profilo personale, le notifiche, lo spazio di lavoro e le aree dedicate agli organizzatori, ai giudici e ai mentor.

La navigazione è inoltre regolata da guardie di routing, come authGuard, guestGuard e roleGuard. Questi controlli permettono di limitare l'accesso alle pagine in base allo stato di autenticazione e al ruolo dell'utente. Ad esempio, l'area organizzatore è accessibile agli utenti autenticati con ruolo ORGANIZER, mentre l'area staff è riservata agli utenti con ruolo JUDGE o MENTOR.

L'architettura a componenti permette di organizzare le pagine come strutture gerarchiche composte da elementi più piccoli, favorendo la separazione delle responsabilità e il riutilizzo del codice. La struttura effettiva delle singole pagine dipende dai componenti utilizzati.

Le guardie Angular intervengono sulla navigazione lato frontend; i controlli di autenticazione e autorizzazione del backend sono comunque presenti e necessari per proteggere le operazioni e le risorse dell'applicazione.